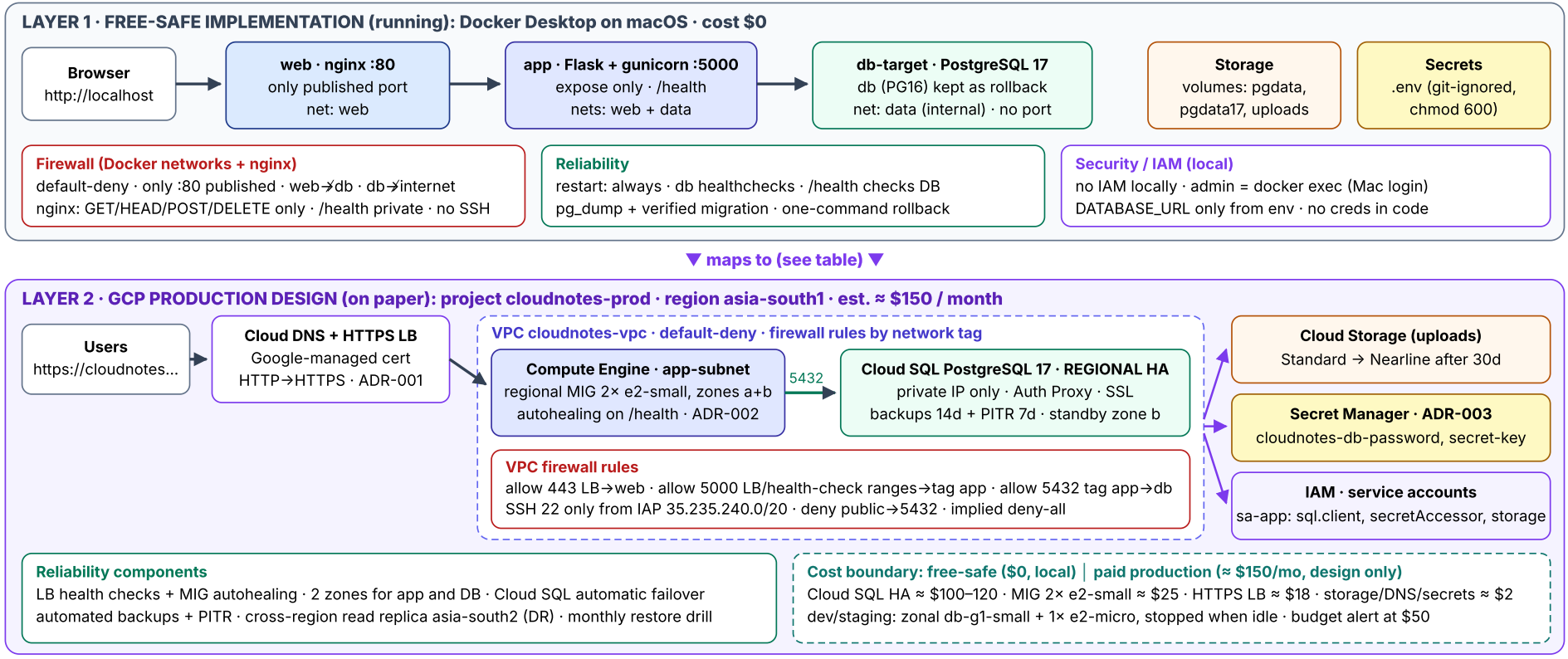
Document 1

Production Architecture Diagram: Free-Safe Implementation → GCP Production

CloudNotes · Layer 1 = what runs today, for free (LU 2.2 / 2.4 / 2.7) · Layer 2 = the GCP production design it maps to · items marked **ADR-00x** are planned improvements from Document 5



Local component (Layer 1)	GCP equivalent (Layer 2)	Why they play the same role
Linux server / Docker Desktop, `restart: always`	Compute Engine regional MIG (systemd / container on each VM)	Both run the same container image; MIG adds autohealing and a second zone
Docker networks + nginx allow-lists (default-deny)	VPC firewall rules by network tag + implied deny	Both decide which tier can reach which port; only the edge is public
PostgreSQL 17 container (`data` network, no port)	Cloud SQL PostgreSQL 17, private IP, regional HA	Same engine and schema; Google runs backups, patching and failover
Docker volume `uploads`	Cloud Storage bucket with lifecycle rules	Attachments leave the VM, so app servers become stateless and replaceable
`.env` file (git-ignored)	Secret Manager, read at runtime by the app service account	Both keep secrets out of code; Secret Manager adds IAM, rotation and audit
Docker bridge networks `web` / `data`	VPC `cloudnotes-vpc` with web / app / db subnets	Both segment tiers into separate private networks
nginx on `http://localhost:80`	Cloud DNS + global external HTTPS Load Balancer	Both are the single public entry; production adds a real name and TLS

Caption: Every local piece has a direct cloud counterpart. The container image, schema and tier boundaries stay the same, and only where each piece runs changes: Docker → Compute Engine MIG, Docker networks → VPC firewall rules, local PostgreSQL → Cloud SQL, volume → Cloud Storage, `.env` → Secret Manager, nginx → Cloud DNS + HTTPS Load Balancer. The dashed cost boundary separates what runs free today from what would be paid in production.

Document 2

Self Architecture Review Checklist: Four Lenses

Reviewed against the running free-safe stack **and** the GCP design pack. Result: **10 PASS** · **10 GAP**. A gap is something that is missing, untested, or only on paper.

🔒 Security

Item	Result	Justification
IAM least privilege	GAP	Only the app's DB/secret roles are designed; no roles for humans, CI or storage, and nothing is enforced locally.
Secret management	GAP	Today the DB passwords sit in plaintext in `.env` on disk (git-ignored, chmod 600). Secret Manager is design only.
Firewall restrictions	PASS	Default-deny, only :80 published, web→db, db→internet. 7/7 connectivity tests passed.
Public exposure	PASS	App :5000 and DB :5432 are unreachable from outside ("Connection refused"); no public DB IP in the design.
Administrative access	PASS	No SSH in any container; designed SSH only via IAP. Note: the `/health` allow-list (172.16.0.0/12) is broader than needed.

🛟 Reliability

Item	Result	Justification
Health checks	GAP	`/health` checks the DB, but only the DB containers have a Docker healthcheck; a hung app is never restarted.
Backup strategy	GAP	Locally, backups are manual `pg_dump` only, with no schedule. Automated backups + PITR exist only in the Cloud SQL design.
Recovery planning	PASS	Migration has a tested one-command rollback (`03-repoint.sh source`), and the source DB is kept intact.
Failover planning	GAP	Single host and single DB today; failover is designed but never tested (no drill).
High availability design	GAP	The DB is regional HA, but the LU 2.2 compute mapping is a single e2-medium VM, so the app tier is a single point of failure.

💰 Cost

Item	Result	Justification
Compute sizing	GAP	e2-medium was picked without load data; a 2-worker app at ~90 MB RAM fits e2-small. Over-provisioned.
Database sizing	PASS	`db-custom-1-3840` for prod, `db-g1-small` for dev, 10 GB with auto-increase. Right-sized for an 8 MB DB.
Storage usage	GAP	Uploads live on a local volume / VM disk; no bucket, storage class or lifecycle rule is designed yet.
Free-safe vs paid boundary	PASS	Everything runs free locally; every paid service is design-only with a cost estimate (≈ $150/mo).
Cost optimization	GAP	No budget alerts, no dev shutdown schedule, no committed-use plan. HA roughly doubles DB cost.

🌐 Networking

Item	Result	Justification
Network segmentation	PASS	Separate `web` and internal `data` networks locally; web/app/db subnets in the VPC design.
DNS architecture	GAP	Reached by `localhost` / bare IP only; Cloud DNS is mentioned but there is no zone or record design.
HTTPS design	GAP	Plain HTTP end to end (browser→nginx and nginx→app). No certificate or HTTP→HTTPS redirect.
Firewall boundaries	PASS	Every tier boundary has an explicit rule, and the dangerous paths are proven blocked by tests.
Private DB connectivity	PASS	No published DB port, internal network locally; private IP + Auth Proxy in the design.

Caption: Strengths: network isolation is real and tested (default-deny, private app and DB, no SSH), data migration and rollback are proven, and the DB is private and right-sized. **Biggest gaps:** no HTTPS or DNS, an app tier that is a single point of failure (single VM, no app healthcheck), secrets in a plaintext `.env` , and backups and failover that exist only on paper. The three most important gaps become the ADRs in Document 5.

Document 5

Improvement Plan: Architecture Decision Records

The three gaps from Document 2 with the highest risk: no HTTPS (security + networking), app tier is a single point of failure (reliability), secrets in plaintext (security)

ADR-001 · HTTPS at the edge with Cloud DNS + HTTPS Load Balancer

Status: Accepted

Context: CloudNotes is plain HTTP end to end. Notes and attachments travel unencrypted, and users reach it only by IP. HTTPS design is a GAP in both the security and networking lenses.

Decision: Put a global external HTTPS Load Balancer with a **Google-managed certificate** in front, add a Cloud DNS A record, and redirect HTTP→HTTPS. Only the LB and health-check ranges may reach the app (tcp:5000). Locally, nginx gets a self-signed cert on :443 for parity.

Alternatives: (1) Let's Encrypt + certbot on the nginx VM: free, but renewal is ours and it ties TLS to one VM. (2) Cloudflare proxy: easy, but adds a third-party dependency. (3) Stay on HTTP: rejected, unsafe.

Trade-offs: + encryption, automatic cert renewal, a single entry point that enables ADR-002, room for Cloud Armor later. – ≈ $18/month for the forwarding rule, TLS ends at the LB (LB→VM traffic stays inside Google's network), and DNS changes take time to propagate.

```
gcloud compute ssl-certificates create cloudnotes-cert \
  --domains=cloudnotes.example.com --global
gcloud compute target-https-proxies create cloudnotes-
https \
  --url-map=cloudnotes-map --ssl-
certificates=cloudnotes-cert
gcloud compute forwarding-rules create cloudnotes-443 --
global \
  --target-https-proxy=cloudnotes-https --ports=443
gcloud dns record-sets create cloudnotes.example.com. \
  --zone=cloudnotes-zone --type=A --ttl=300 --rrdatas=
<LB_IP>
```

ADR-002 · App tier as a regional Managed Instance Group with autohealing

Status: Accepted

Context: The DB is regional HA, but the app is one e2-medium VM: a single point of failure, sized without data, and with no app healthcheck. A zone outage or a hung process takes CloudNotes down.

Decision: Run the container on a **regional MIG of 2× e2-small** across zones a and b, behind the ADR-001 LB, with **autohealing** on `/health` and autoscaling to 4 at 70% CPU. Move uploads to a Cloud Storage bucket so the VMs are stateless. Locally, add a Docker healthcheck to `app`.

Alternatives: (1) One bigger VM: simple, but still a single zone. (2) Cloud Run: serverless and cheap at low traffic, but needs a Serverless VPC connector to reach private Cloud SQL and a code change for uploads. (3) GKE: too heavy for one small service.

Trade-offs: + survives a zone failure, replaces hung VMs automatically, and 2× e2-small (≈ $25) costs about the same as one e2-medium. – more moving parts (template, health check, LB), the app must stay stateless, and rolling updates are needed for deploys.

```
gcloud compute health-checks create http cloudnotes-hc \
  --port=5000 --request-path=/health --check-
interval=10s
gcloud compute instance-groups managed create
cloudnotes-mig \
  --region=asia-south1 --zones=asia-south1-a,asia-
south1-b \
  --template=cloudnotes-tpl --size=2 \
  --health-check=cloudnotes-hc --initial-delay=60
gcloud compute firewall-rules create allow-hc-to-app \
  --network=cloudnotes-vpc --allow=tcp:5000 --target-
tags=app \
  --source-ranges=130.211.0.0/22,35.191.0.0/16
```

ADR-003 · Secrets in Secret Manager with per-service least-privilege IAM

Status: Accepted

Context: DB passwords and `SECRET_KEY` sit in plaintext in `.env` on the server. Anyone with disk or backup access can read them, there is no rotation or audit trail, and IAM for humans/CI isn't designed (two security GAPs).

Decision: Store `cloudnotes-db-password` and `cloudnotes-secret-key` in **Secret Manager**. The app's service account `sa-app` gets `secretAccessor` **on those two secrets only**, plus `cloudsql.client` and `storage.objectUser` on the uploads bucket. Humans get viewer roles; deploys use a separate `sa-deploy`. Rotate every 90 days.

Alternatives: (1) Keep `.env` with chmod 600: free, but no audit or rotation, and copies leak into backups. (2) Instance metadata: readable by anything on the VM. (3) HashiCorp Vault: powerful, but we'd have to run and secure it ourselves.

Trade-offs: + no secrets on disk or in images, IAM-gated access, audit logs, versioned rotation. – app startup depends on the Secret Manager API, ≈ $0.06/secret version/month plus access costs, and more IAM bindings to maintain.

```
gcloud secrets create cloudnotes-db-password \
  --replication-policy=automatic --data-file=-
gcloud secrets add-iam-policy-binding cloudnotes-db-
password \
  --member="serviceAccount:sa-app@cloudnotes-
prod.iam.gserviceaccount.com" \
  --role="roles/secretmanager.secretAccessor"
# at startup on the VM (no password on disk):
DB_PASS=$(gcloud secrets versions access latest \
  --secret=cloudnotes-db-password)
```

Backlog (not ADRs yet): automated local `pg_dump` cron + a monthly restore drill · budget alert at $50 · Cloud DNS zone design · narrow the `/health` allow-list · a failover drill on staging.

Caption: Together the three ADRs close the riskiest gaps. ADR-001 encrypts all traffic and gives CloudNotes a real name and one controlled entry point. ADR-002 removes the last single point of failure by running the app in two zones with automatic healing, at roughly the same cost. ADR-003 takes secrets off disk and puts every credential behind least-privilege IAM with audit and rotation. Each records the alternatives and the cost it accepts, so the decisions can be revisited when requirements change.